Title & Summary

No Cameraman Left Behind

The idea is to allow cameraman place himself within a group of people/objects in a picture. The user of this project should be able

- to select the cameraman from a day-to-day environment which is complex and may have many other humans in the frame, and then
- insert the selected 'cameraman' into an environment which is as complex and typically occuring in a day-to-day life.

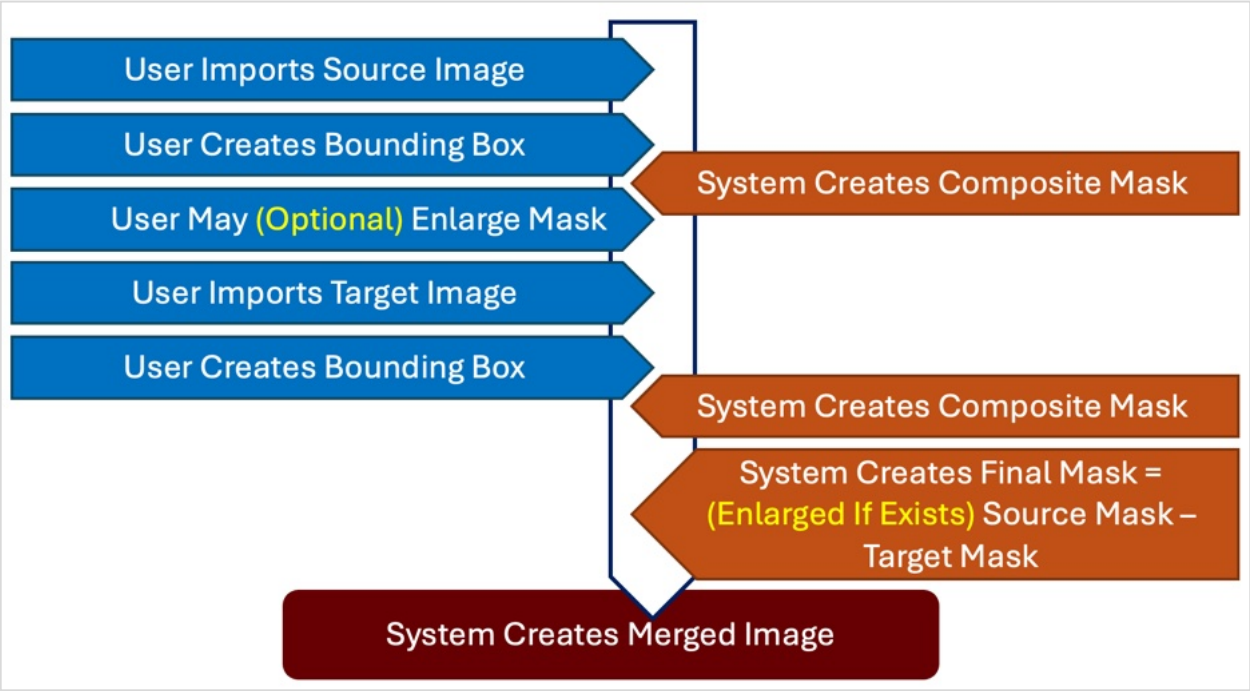
The problem trying to solve and for whom

Cameraman Inclusion: A project that enables a cameraman (may be a family member taking a simple posed group picture or a journalist who may later want to insert themselves into the scene they are reporting about) to easily insert themselves in a complex setting, while selecting objects they want to be in the front and objects to be behind themselves, using advanced AI techniques (e.g., image segmentation, image blending).

Inserting two humans behind the human in the target in a plane background.	Inserting the source behind the target objects in slightly complex background
Inserting the source behind the target objects in a day-to-day typical environment.	Inserting the source behind the target objects in a day-to-day typical environment.

Solution

- Used Google Colab & Jupyter notebook
    - Jupyter BBox Widget for user interaction to create the bounding boxes, i.e., interactive user experience, provide a user-friendly interface (on Google Colab).
- Meta SAM2, mask predictor model for image segmentation
- OpenCV's linear interpolation (Poisson's) or CV2 seamlessClone for image blending



Pipeline and Baseline Results

Pipeline

- **Image Segmentation using Meta's SAM2 Model**:
  - Use Meta's SAM2 (Segment Anything Model 2) to perform automatic image segmentation on both the cameraman's photo and the group image.
  - SAM2 will generate segmentation masks of the people and objects in the images.
  - For the cameraman's photo, SAM2 will segment out the cameraman cleanly so they can be extracted from the background.
- **Gradient Blending using OpenCV, linear interpolation**
- Application Deployment (Colab Notebook as Interface):
  - Create a simple interface using **Google Colab** widgets for users to upload images.
  - **Jupyter BBox Widget** for user interaction to create the bounding boxes

Baseline results

- **Misplaced source figure** in the blended image, e.g. as shown, the inserted human figure is floating at the center due to wrong center position determined by the **CV2.seamlessClone**.
- **Poor & pixelated image segmentation**, not selecting complete human figures, as shown in the image.
- When composing the mask using the source object and mutliple target objects behind which the source object was to be put, I realized the challenge of picking the appropriate **enlargement size around the mask**.
- **Unwanted source background** around the source figure in the blended image
- **Fuzzy edges** in the source figure in the blended image

Experiments & results

Experiments in a nutshell:

- **Image Segmentation & Mask Creation using Meta's SAM2 Model**:
    - Among the two models: **Automatic Mask Detector** and **Predictor Mask**, I used Predictor Mask so the user can select their source object. Further, then selection was to be made to what fidelity of model to be used among '**tiny**', '**small'**, '**base plus**' and '**large**', considering the cost of GPU. Then how to create a composite mask of only the objects of interest from the image.
    - Results with mask hugging the selected object.
    - Results with mask enlarged around the selected object.
    - Results with enlarging the source-mask before subtracting the composite object-mask.
    - Blackening/graying source image outside the source-mask.
    - Preprocessing **mask into grayscale** or **three color channels**.
    - **Contour detection** using cv2.findContours to find the contours in the binary mask to help identify the area of interest.
    - Modifying the mask by changing all black pixels outside the area of interest to gray using **np.where**.
- **Object selection**: **Jupyter BBox Widget** for user interaction to create the bounding boxes.
- **Pose Adjustment with ChatPose**: https://yfeng95.github.io/ChatPose/
    - It didn't work for me because I didn't have an UI for user to provide directions using NLP. I tried to dig deeper and couldn't figure out how to use ChatPose with Google Colab.
- **Blending Algorithms**
    - **Adobe Unihuman**: I researched into it since this was suggested by the professor, but couldn't find a way to use Adobe Unihuman with Google Colab.
    - **Stable Diffusion and impainting in Colab**
        - **Limited experimentation due to GPU costs.**
    - **CV2 seamlessClone**: I used this for my initial experiments and it worked well when I had a mask which was simple, for one object and not a composite mask which I needed. I didn't use it in favor of Poisson's, but later realized that CV2 seamlessClone also was as good as an alternative.
        - **Issues with seamlessClone() center determination**- the center determination by the seamlessClone() was misplacing the source ojbect during insertion in the target image.
    - **OpenCV linear interpolation** or Poisson's equation: Finally I ended up using this. In trying to correct for my purpose, I experimented with tightening the mask, feathering the mask edges, enhancing gradient computation or clamping gradients, edge padding in mask, blending intensities at the boundary etc.
- Application Deployment (Colab Notebook as Interface)

Experimental Journey In Detail:

Installing Meta's SAM2 Image Segmentation

I had a tough time getting SAM2 working. I tried to use Google Colab associated with upenn account. After much stress and time, I realized that unless I will **purchase credits for GPU with Ampere (8.0) CUDA** capability, it will not work.

I tried to explore other options such **Cloud Run, GKE, or Vertex** with new google account (because as per the Ed post of a student, Google provides $300 credit with a new account), but after some exploring into it, decided to continue with Colab itself.

And above all, the **Upenn Google Colab account blocked purchasing GPUs**, so I had to use my personal Google account. Figuring all of this took quite a amount of time to get image segmentation part set.

Automatic Mask Generation Using Meta's SAM2 Model

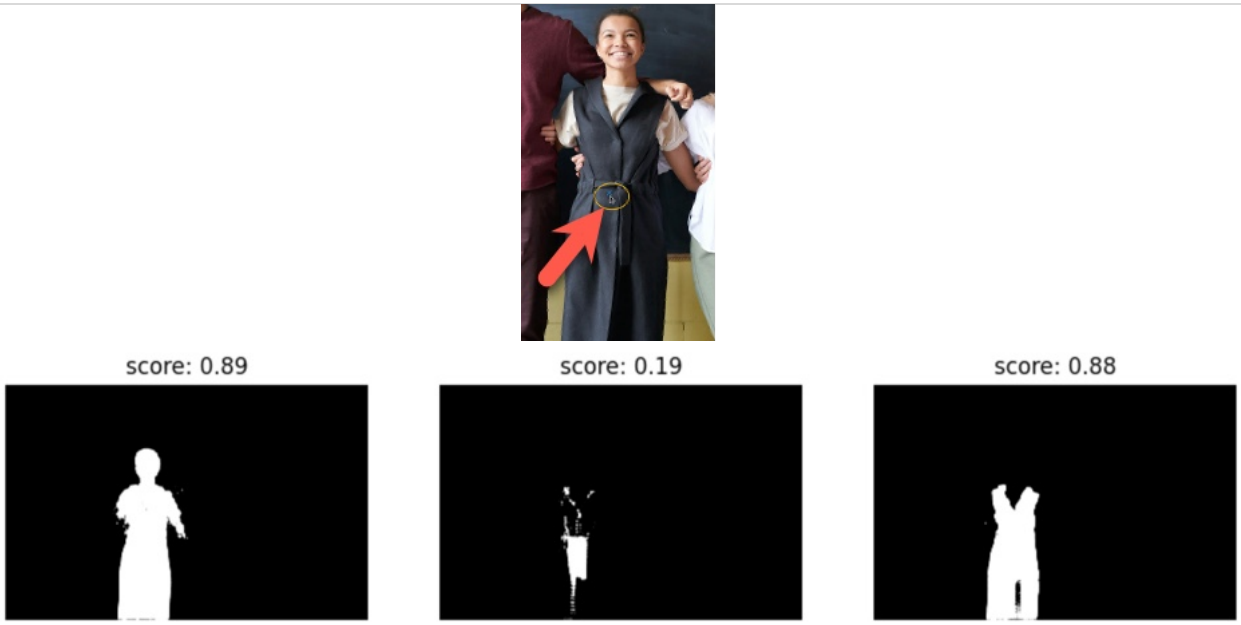
SAM2 had two possibilities to segment images:

- Automatic Mask Generation
- Mask Predictor

I tried the Automatic Mask Generator first. It gave a range of possibilities, from segmenting the image into each discrete possbile individual masks as shown below, even segmenting different pieces of clothing etc. to more composite masks, such as each whole individual human.

Original image	Multiple masks generated

⇒ This was not useful because the auto mask-generation created multiple masks, even segmenting different pieces of clothing on each human.



Automatic Masks with different scores in SAM2 are created

- ⇒ It provided different masks with different scores, from segmenting whole human to even breaking them down further.
- ⇒ This was very unpredictable based on individual image.
- ⇒ I **rejected** it because it depended haevily on user's judgement to tweak the score to get the right mask.

SAM2's Mask Predictor

This offered selecting an object by drawing a bounding box or by clicking.

User selection by clicking on the human figure

Original image	User clicks on its chosen human-object

- ⇒ This was better than auto-mask generation but then the user still had to provide a score for SAM2 to create a mask, which may not be accurate for the need.

User selection with a bounding box

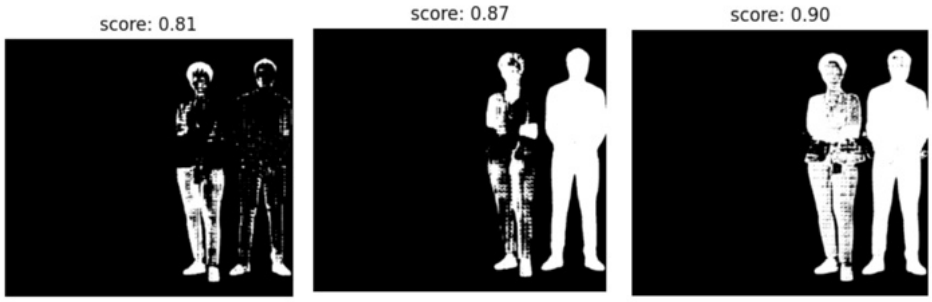
User creates a bounding box around its chosen human-object	Chosen human is segmented

- ⇒ **This was the best for my requirement, as the user will be able to pick the photographer amid other distractions in the background.**
- ⇒ I learned about and used (from **jupyter_bbox_widget** import **BboxWidget**) to create bounding box, which worked well on Google Colab.

Segmentation Models: Tiny, Small, Base Plus, Large

I experimented with all the four models and found that 'Tiny' model was adequate for my purpose.

Multiple Humans Selection for Mask Creation with Single Bounding Box

	
Multiple selection with a bounding box	Result was a pixelated poor quality mask, and even with different scores, it was unpredictable

- ⇒ Multiple object selection with a single bounding box in an image resulted in a poor mask.

Single Human with a Bounding Box

Selection by Bounding box

Segmented image

Mask

⇒ As noticed earlier, a single object within a bounding box in any image gave a good mask.

⇒ **Thus the solution was to use multiple bounding boxes and then combine the masks into one composite mask**.

Combined Mask Creation

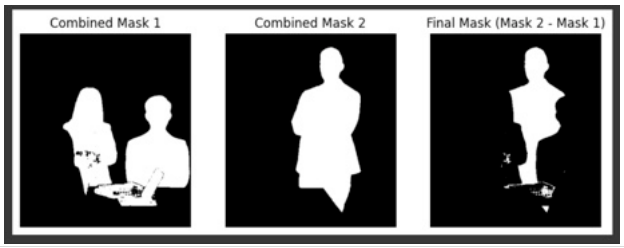
Three bounding boxes generated clean individual masks for laptop, human figures

Composite Mask

⇒ **This was a very successful breakthrough to generate a composite mask.**

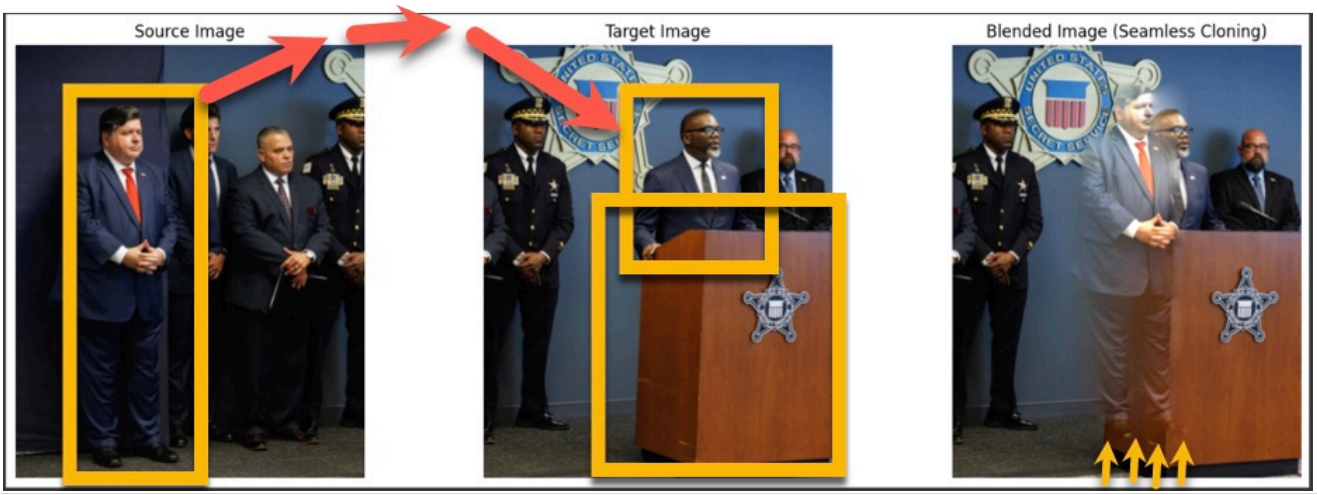
Composite Mask Creation (Source – Target)

To place the source object behind certain objects in the target image, I wanted to create a composite maski, i.e. **Source – Target**.

	
Goal is to insert the source object behind a few target objects	A combined mask of target objects behind which the source object will be placed, and then a composite mask is created by subtracting Source - Target

⇒ **A composite mask created by Source Mask – Target Combined Mask.**

Image Blending by CV2.seamlessClone



The source position in the blended image is wrong

The problem I am having is that it putting the center of the mask-white-area in the center of the target image, instead of placing the mask+target wherever it is. E.g. the dimensions of the images and masks as well, are image_bgr.shape=(497, 400) and the center of mask is (200, 248). But I have to manually put it off-center as (115,280) to get it at the right position as below. Secondly it is still showing more than wanted blending with the background, even though I have binarized the mask.

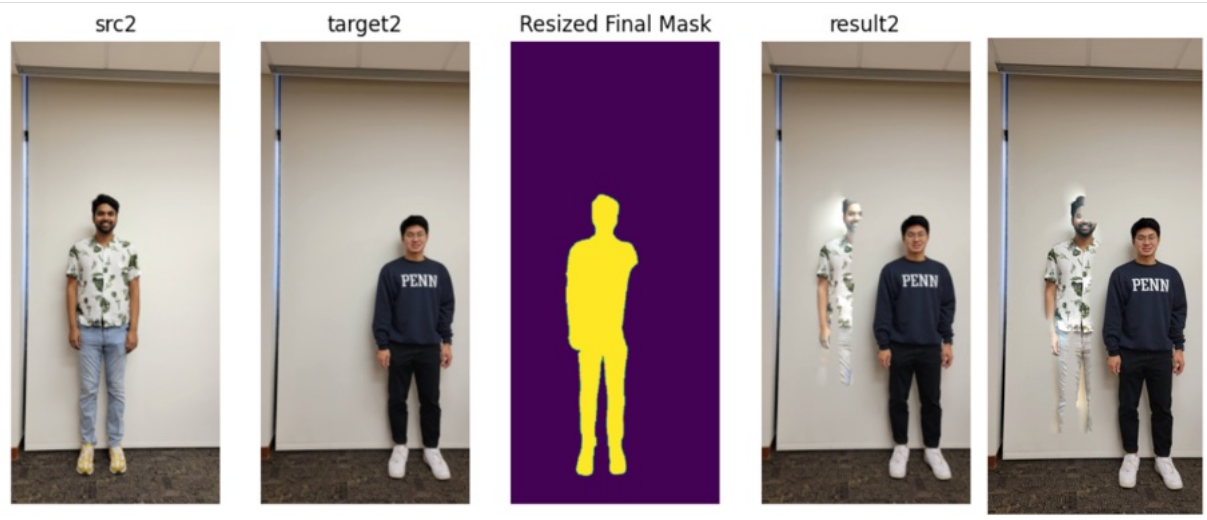
- ⇒ The center position of the mask-image whenever was used by the **cv2.seamlessClone** was assuming it to be center of the white portion of the mask, as evident in the final image shown, where the source is lifted off the floor to the center of the blended image.
- ⇒ **This was a drawback, which would put a lot of responsibility on the user to determine the 'corrected center'**.

Image Blending by Poisson Equation

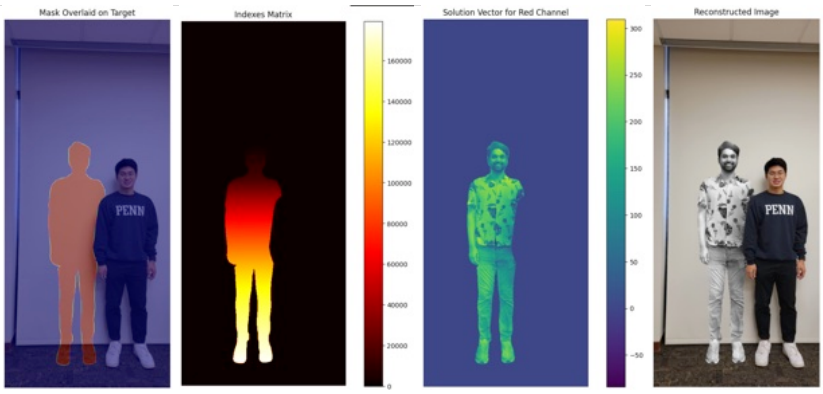
The image blending by Poisson Equation solved the positioning of the source object.

Next, I faced a series of challenges because of the composite mask, i.e. trying to put one object behind some.

At first I was reducing the images to be appropriately displayed within the Colab cell, so that the user don't have to scroll to draw bounding boxes. And then once the mask was created, I was resizing the mask to the original image size. This was causing a lot of miscalculations and weird results.

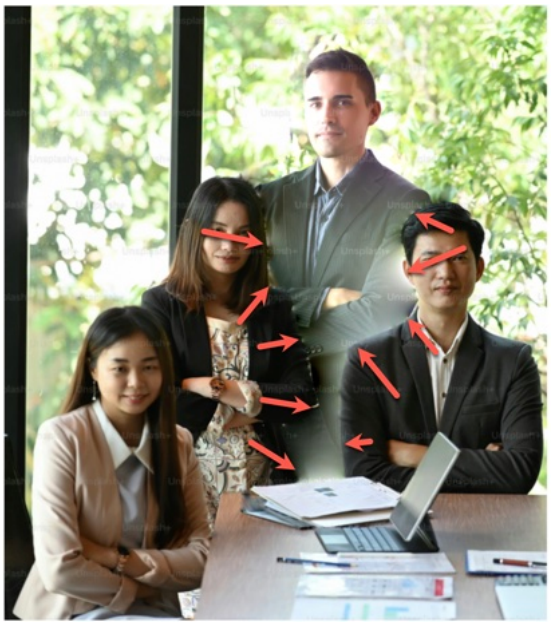


The final image **was misplacing the source image**, which sent me through a lot of debugging before I could figure out what was happening.



⇒ **As a solution, I didn't scale down the original images, but just displayed them to smaller scale to facilitate creating bounding boxes, which the user created on the original sized images.**

Other experiments

	
Target background bleeding from behind the 'target-mask' into the source.	Darkened source image.

Ghosted source image, when the mask is hugging the segmented images	**Enlarged mask bringing halo over the source human from the source-background**
So, then tried making the source image all black wherever the mask = black. **Well, it removed the reflections, but kept the target image darkened then it was**.	Making with mask '0' values as '128'. **The target image is darkened**.

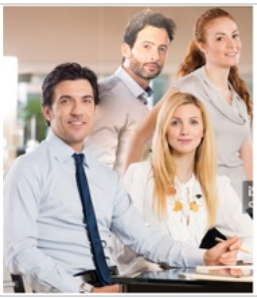
	It turns out that if the mask is completely tight the border gradient is then artificially interpreted as null. That is why it ignores it completely and produces ghosts. Using a larger mask by dilating the original mask using morphological operations and therefore including some background is thus the solution. Care must be taken when choosing the color of the background included if the contrast is too big the gradient will be too strong and the image would not be well blended. Using a color like gray is a good starting point.

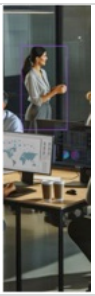
I tried to deal these issues with trying out:

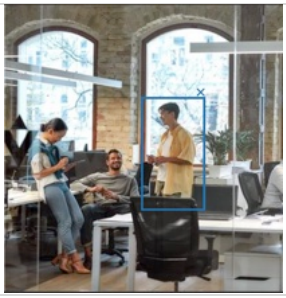
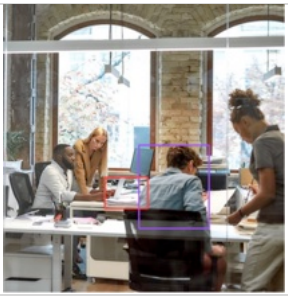
1. **Preserve Source Object's Pixels:** When computing the gradients in the getSolutionVect function, giving priority to the source image's content to prevent target background "bleeding" into the source object.
2. **Refine Mask with Edge Padding:** If the target background is bleeding into the source object, ensuring the mask includes a slight edge padding of the source object. it created a smoother boundary by expanding and blending edge pixels of the source object into the target.
3. **Blend Intensities at the Boundary:** Instead of a binary mask, using a soft alpha mask that blends the source and target intensities at the boundary to minimize hard transitions
4. **Clamp Gradients:** To limit the effect of the target background, clamping the gradient values to a range that matches the source object's intensity range.
5. **Gradient Balancing**: In getSolutionVect, the source gradient and target boundary contributions are averaged (/2) to prevent one from overpowering the other.
6. **Clipping Output Values**: In reconstructImg, pixel values are clipped to the range [0, 1] to avoid over-brightening or artifacts.
7. **Normalization**: Throughout the pipeline, ensure normalization steps are consistently applied to maintain color fidelity.

Final Results

Source	Target	Blended

		
Source	Target	Blended

		
Source	Target	Blended

		
Source	Target	Blended

Project timeline & Milestone & duties in group (1 pt)

I couldn't get a partner, hence I **worked alone**.

I will followed the project timeline and milestones as given in the 'Final Project Problem sheet'.

- 10/25-Making all pieces of the pipeline to work in a streamline manner.
- 11/1-Multiple iterations in polishing each part of the pipeline to provide richer capability.
- 11/7- Midterm Progress Report submission
- 12/5- Final Report submission

What further work would you do if you had more time (1 pt)

- I would like to work in a group. This time, I **couldn't find a partner** though I had tried. That would have made things much easier to brainstorm, share the load and cost.
- **Cost of using SAM2** was a severe limitation for me. I was continuously worried about the growing expense, each time I had to recharge for more use. Eventually, I had to stop recharging and be satisfied with the results for the scope of this project.
- Another limitation I felt was hesitation in using any personal pictures. I used **stock pictures** and they often had stamp on them, available in low-fidelity for free, hindering good mask creation or they were not very suitable for my purpose.

Having said that, if I had more money and time, I would:

- Pose Adjustment with **ChatPose**:
    - Explore ChatPose to adjust the cameraman's pose to match the group's orientation, to ensure that the cameraman's posture, head tilt, or body direction aligns with the group, preventing awkwardness in the final photo.
    - It will be fun to use ChatPose utilize natural language commands or learned pose suggestions to refine the cameraman's posture.
- Explore and use HuggingFace's **Stable Diffusion**, which I found how I could use in Google colab eventually.
- Blending Algorithm (Adobe Unihuman):
    - I would try seamlessly blending the cameraman into the group photo using **Adobe's Unihuman** technology. This **AI-based blending algorithm** which is designed for **photorealistic** integration of subjects, **handling lighting**, **shadows**, and **texture inconsistencies** would have provided amazing realistic results.
- I would like to refine the **mask creation** and **image gradient** process such that blended image is much more automated using may be **CNN**, than requiring user intervention and realistic in terms of orientation and light effects.
    - Currently, the user at times will have to tweak with **tightening the mask**, **feathering the mask edges**, **enhancing gradient computation** or **clamping gradients**, **edge padding** in mask, **blending intensities at the boundary** etc. to get some images correct.

What did you learn from this project experience? (2 pts)

I learned the following:

- **Google Colab** is not the best environment for trying to create a very refined product using existing APIs because a limited tech stack can be practically used with it.
- Using tools like **SAM2** for image segmentation instead of other alternatives was a mistake for my project scope. It definitely gave me the benefit of not having to use **contours** etc., but in hindsight, I should have used something which would have worked without purchasing **A100 GPU**. SAM2 necessarily needed A100 GPU or above.
- It was fun to explore SAM2's two mask generators:
    - **Automatic_mask_generator**
    - **Sam2_image_predictor**, which I ended up using with the user's drawn bounding box. SAM2 was pretty good in isolating an object even with the most basic model, i.e. '**tiny**', others were '**small**', '**base_plus**' and '**large**', which would have consumed more GPUs.
- To let the user select his source object and target objects, I learned about **Supervision** and **Jupyter BBox Widget**, which allows to create bounding boxex within Jupyter Notebook environment.
- I learned to **combine masks** and **deduct one mask** from another.
- I learned that using a single mask is easier to use for image blending, but using a **composite mask** to get for layered image blending is tricky and complex. My code worked better when the background was flat, but with complex background the success was 50% and needed user to be an expert to tweak the mask size etc. to correct the failed 50%. In trying to correct, I experimented with **tightening the mask**, **feathering the mask edges**, **enhancing gradient computation** or **clamping gradients, edge padding** in mask, **blending intensities** at the boundary etc.
- I learned about **cv2.seamlessClone** and more deeply about using OpenCV, to blend images using weighted sums, **Gaussian pyramids**, **Poisson equation**.
- As I worked on refining the blended image, I learnerd about preserving source object's pixels when desired, refind mask with edge padding instead of flatly enlarging it, belnding intensities at the boundary or clamping the gradient to limit the effect of the target background.

Recorded video of project results / demo (10 pts)

https://drive.google.com/drive/folders/1yhq7yyUQkve0fnwkKKoMDGg3ztfxUvB8?usp=sharing

Project code (5.0 pts)

https://drive.google.com/drive/folders/1yhq7yyUQkve0fnwkKKoMDGg3ztfxUvB8?usp=sharing